



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ FACULTAD DE CIENCIAS

"Sistema Automatizado
de Estacionamiento."

Laboratorio de Instrumentación
Virtual

Alumno
Emilio Martínez Mendoza

1 de junio del 2026

1. DESCRIPCIÓN DEL SISTEMA

El presente proyecto consiste en el desarrollo de un sistema inteligente para la gestión automatizada de un estacionamiento. El objetivo principal del sistema es monitorear la disponibilidad de los cajones en tiempo real y digitalizar el proceso de cobro mediante boletos virtuales.

Para garantizar la estabilidad del proyecto, el software se diseñó dividiendo las tareas en tres áreas tecnológicas:

- **Visión Artificial:** Utilizando Python y el modelo de inteligencia artificial YOLO para detectar los vehículos mediante cámaras.
- **Interfaz Industrial:** Utilizando LabVIEW como el punto de venta donde el usuario interactúa, ve los lugares disponibles y se calcula su tarifa.
- **Conectividad y Nube:** Integrando herramientas de red para que el sistema local pueda respaldar datos en internet y el cliente pueda acceder a su ticket desde su dispositivo móvil.

Esta separación de tareas permite que el procesamiento matemático pesado no congele la pantalla de cobro del usuario, resultando en un sistema de nivel comercial rápido y robusto.

2. IMPLEMENTACIÓN DE APIs

En el desarrollo de este proyecto, las diferentes tecnologías utilizadas (Python, LabVIEW y los dispositivos móviles de los usuarios) operan de manera independiente y no pueden intercambiar información de forma directa. Para solucionar este obstáculo, se implementó una API.

De manera sencilla, la API funcionó como un puente, su trabajo fue recibir una solicitud de un programa, buscar la información necesaria y entregar la respuesta de vuelta. En el sistema, la API fue construida utilizando el servidor local Flask y cumple con dos tareas vitales:

- **Comunicación Interna:** Permite que la pantalla de LabVIEW "consulte" constantemente al motor de Python para saber cuántos cajones de estacionamiento están disponibles. La API toma las coordenadas de los vehículos detectados por la cámara y se las entrega a LabVIEW empaquetadas en un formato json.

- **Comunicación Externa (Nube y Usuarios):** Permite que el sistema se conecte hacia el exterior. Gracias a la API, la información de los boletos se guarda automáticamente en la base de datos de Google Sheets. Además, es la encargada de recibir las peticiones de los celulares de los clientes cuando escanean el código QR, entregándoles su boleto virtual dinámico y gestionando el envío de correos electrónicos.

3. DIAGRAMA DE FUNCIONAMIENTO

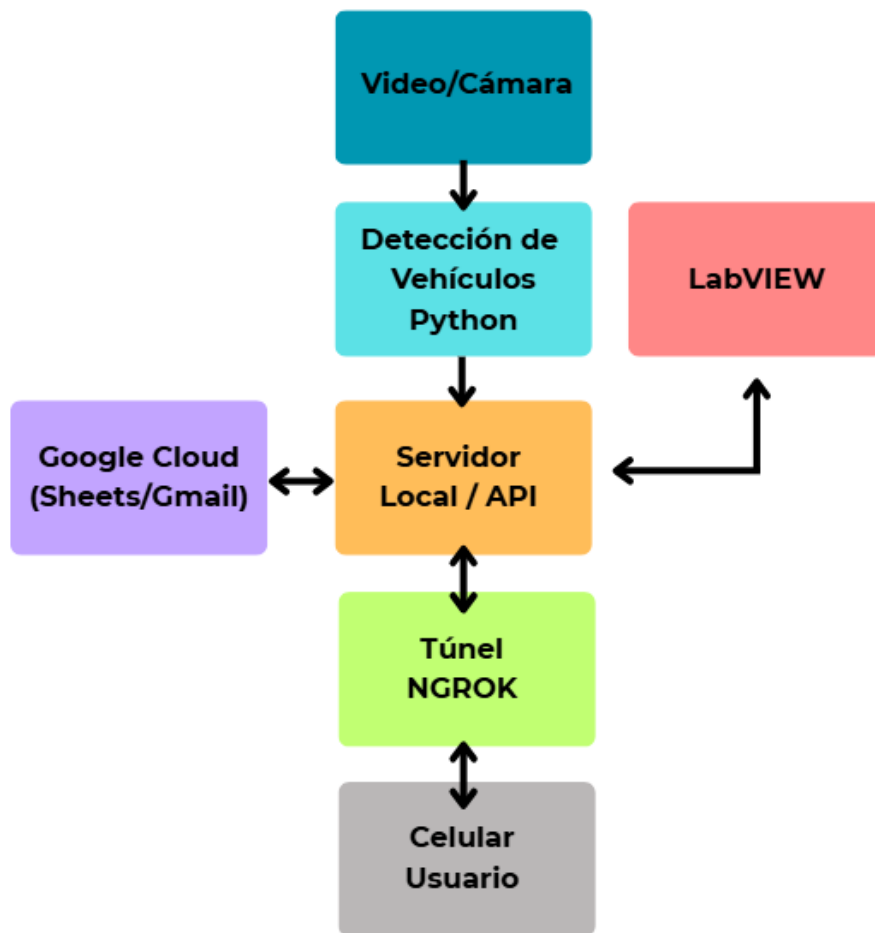


Figura 1. Diagrama de Funcionamiento General del Sistema.

El diagrama ilustra la arquitectura de comunicación del proyecto, la cual está estructurada alrededor de un servidor central. El sistema opera mediante la interacción de los siguientes módulos:

- **Adquisición y Procesamiento** El flujo inicia con la captura de video en tiempo real. Esta señal es procesada por el motor de visión en Python (modelo YOLO), el cual se encarga exclusivamente de la detección de los vehículos y la extracción de sus coordenadas.
- **Servidor Local / API:** Las coordenadas generadas por la inteligencia artificial se envían al servidor local construido con Flask. Este servidor reúne los datos y gestiona la comunicación bidireccional con el resto de las plataformas.
- **Interfaz de Usuario (LabVIEW):** Se comunica constantemente con el servidor local mediante peticiones HTTP. Extrae la información de ocupación y los tiempos de entrada para calcular tarifas y actualizar la pantalla gráfica de manera fluida.
- **(Google Cloud):** De manera simultánea, el servidor de Python se comunica con los servicios de Google para respaldar el historial de los tickets en una base de datos (Sheets) y automatizar el envío de notificaciones y mapas mediante Gmail.
- **Acceso del Usuario (ngrok y Celular):** Para la visualización de los tickets virtuales, se implementa un túnel de red (ngrok). Esto permite que el servidor local reciba peticiones externas de forma segura, logrando que el cliente pueda escanear su código QR.

4. DESCRIPCIÓN DEL FLUJO DE DATOS

Fase 1: Inicialización del Entorno y Conexión Backend

El flujo de datos comienza en el panel principal de LabVIEW. Al presionar el botón de inicio de la interfaz, el sistema ejecuta un comando automático hacia la terminal del sistema operativo (CMD) que enciende el servidor de Python en segundo plano. Esto establece la comunicación bidireccional y pone al sistema en *While Loop*.

Fase 3: Adquisición de Datos y Mapeo Visual

Una vez iniciada la comunicación, el modelo de Inteligencia Artificial comienza a detectar los vehículos y actualiza las coordenadas en un archivo JSON local. LabVIEW ejecuta un ciclo *For* que lee continuamente este archivo para extraer las posiciones y así poder mostrar en un indicador los vehículos detectados.



Figura 4. Interfaz principal de LabVIEW que muestra los vehículos actuales en el estacionamiento.

Fase 4: Registro de Ingreso y Generación de Boleto

Cuando un nuevo usuario requiere acceso, el operador selecciona la opción de "Imprimir Ticket". En ese instante se desencadenan dos eventos, LabVIEW genera visualmente un código QR con un identificador único (ID).



Figura 5. QR desplegado en la interfaz 1.

Y de manera paralela, la API envía un paquete de datos hacia Google Cloud. La base de datos en Google Sheets registra el ID del boleto, captura el *timestamp* (hora exacta de llegada) y asigna el estado inicial como "PENDIENTE".

	A	B	C
1	Generación de Boleto	ID	Estado
2	2026-05-31 16:55:23	QR-568B	PAGADO
3	2026-05-31 19:52:12	QR-3A53	PENDIENTE
4	2026-05-31 20:12:44	QR-E5C8	PAGADO
5	2026-05-31 20:23:54	QR-9132	PENDIENTE
6	2026-05-31 20:32:43	QR-CB42	PENDIENTE
7	2026-05-31 20:33:35	QR-9A5D	PAGADO
8	2026-05-31 20:36:33	QR-7BFD	PAGADO
9	2026-05-31 20:39:00	QR-5D23	PENDIENTE

Figura 6. Sincronización en la nube (Google Sheets) registrando el ingreso del vehículo.

Fase 5: Portal Web y Experiencia Móvil

El usuario escanea el código QR impreso utilizando la cámara de su dispositivo móvil. A través del túnel de red (ngrok), el celular del cliente se conecta de forma segura al servidor local. El sistema responde entregando una página web dinámica que despliega la información de su ticket virtual, incluyendo su hora de entrada y su identificador.



Figura 7. Despliegue del boleto virtual en el dispositivo del usuario.

Fase 6: Sistema de Notificaciones Cloud

Para designar los lugares libres el algoritmo de LabVIEW cruza las coordenadas de cada vehículo detectado con el área geométrica de cada cajón de estacionamiento, si detecta una intersección, el lugar cambia su estado visual a color rojo (ocupado) en la vista aérea de la interfaz.

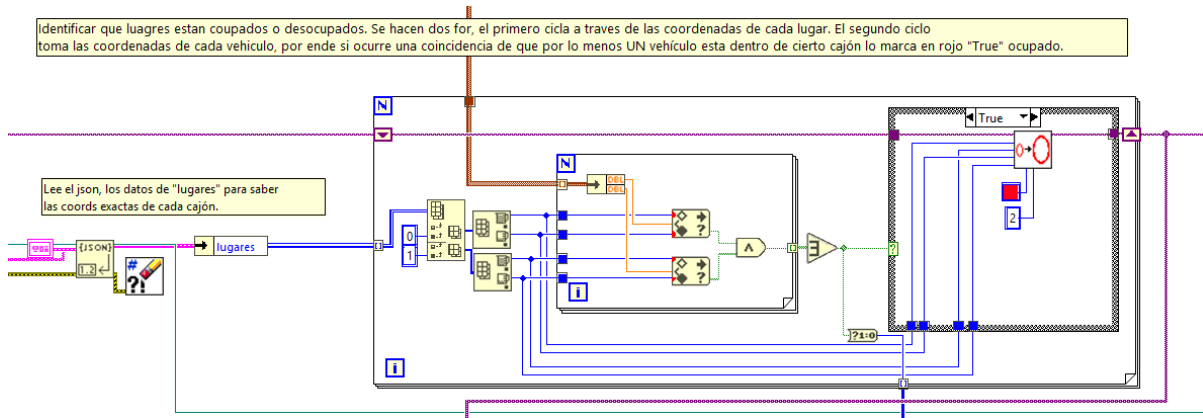


Figura 8. Lógica de programación para designar si un lugar está ocupado o desocupado.

Dentro del portal web del boleto, se ofrece al usuario la opción de solicitar un mapa de lugares disponibles. El cliente ingresa su dirección de correo electrónico en el formulario de la página y envía la petición. La API de Flask recibe este parámetro mediante una petición HTTP POST y ejecuta un script de Google Cloud que envía automáticamente un correo electrónico a la bandeja de entrada del usuario con la información gráfica de los lugares desocupados.

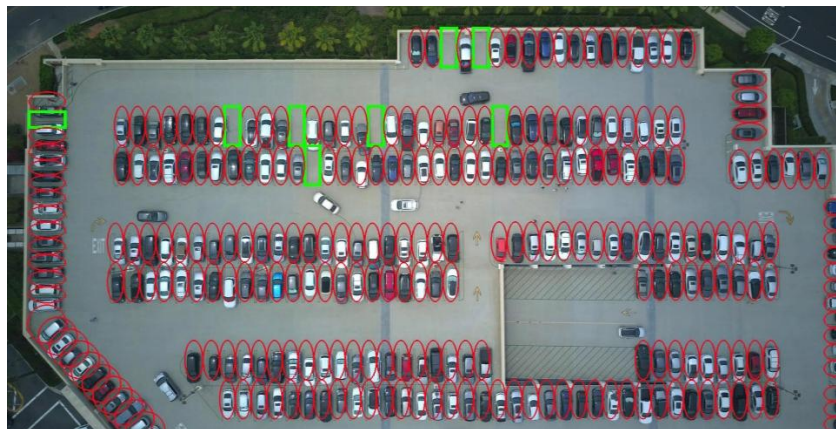


Figura 9. Imagen que llega al correo, mostrando la disponibilidad de lugares.

Fase 7: Escaneo de Salida y Lectura Óptica

Para iniciar el proceso de liberación y pago, el sistema cuenta con un segundo panel de LabVIEW diseñado para la caseta de cobro. El usuario muestra su código QR a una cámara conectada físicamente al sistema. Mediante un módulo de visión y decodificación de LabVIEW, el programa lee la matriz del QR y extrae el identificador único del boleto para buscarlo en la red.

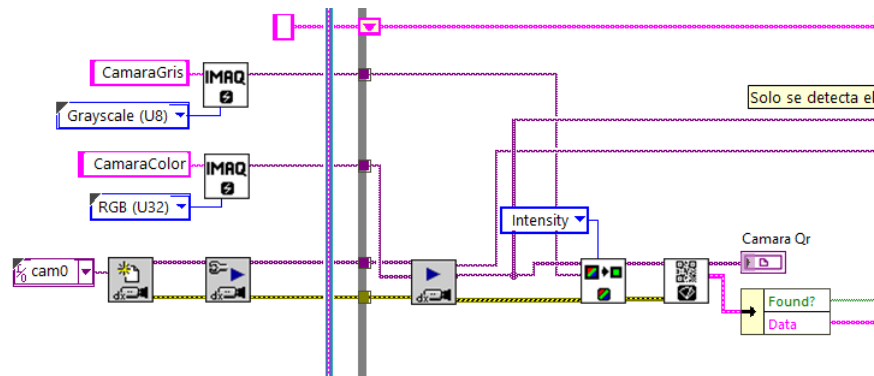


Figura 9. Módulo para decodificar el QR en Labview.

Fase 8: Cálculo Dinámico y Cierre de Transacción

Con el identificador extraído, el sistema local consulta la base de datos de Google Sheets para recuperar la hora de entrada de ese cliente específico. LabVIEW realiza una resta entre el *timestamp* actual y la hora de ingreso registrada, calculando el tiempo total transcurrido. Con base en este tiempo, se despliega la tarifa dinámica en pantalla.



Figura 10. Interfaz 2, muestra el QR, el tiempo que se estuvo en el estacionamiento y el total a pagar.

Una vez confirmado el pago, el sistema realiza una última petición HTTP para sobrescribir la celda en Google Sheets, cambiando el estado de "PENDIENTE" a "PAGADO" y cerrando el ciclo de vida de los datos.

1	Generación de Boleto	ID	Estado
2	2026-05-31 16:55:23	QR-568B	PAGADO
3	2026-05-31 19:52:12	QR-3A53	PENDIENTE
4	2026-05-31 20:12:44	QR-E5C8	PAGADO
5	2026-05-31 20:23:54	QR-9132	PENDIENTE
6	2026-05-31 20:32:43	QR-CB42	PENDIENTE
7	2026-05-31 20:33:35	QR-9A5D	PAGADO
8	2026-05-31 20:36:33	QR-7BFD	PAGADO
9	2026-05-31 20:39:00	QR-5D23	PENDIENTE

1	Generación de Boleto	ID	Estado
2	2026-05-31 16:55:23	QR-568B	PAGADO
3	2026-05-31 19:52:12	QR-3A53	PENDIENTE
4	2026-05-31 20:12:44	QR-E5C8	PAGADO
5	2026-05-31 20:23:54	QR-9132	PENDIENTE
6	2026-05-31 20:32:43	QR-CB42	PENDIENTE
7	2026-05-31 20:33:35	QR-9A5D	PAGADO
8	2026-05-31 20:36:33	QR-7BFD	PAGADO
9	2026-05-31 20:39:00	QR-5D23	PAGADO

Figura 11. Actualización en la base de datos después de haber pagado.



Figura 12. *Interfaz para el usuario completa.*